

# Fixing macOS "any" Interface Error

<https://gemini.google.com/app/14d11fb92fd9025b>

```
User prompt: (base) → cpp git:(main) sudo ./build/tlsfp_engine -i any -f "tcp port 443" [*] Listening on interface: any [*] Capture initialized.
Datalink type: 12 [-] Warning: Redis database unavailable; fingerprints will be reported as unknown. (base) → cpp git:(main) sudo
./build/tlsfp_engine -i en0 -f "tcp port 443" -v Password: [*] Listening on interface: en0 [*] Capture initialized. Datalink type: 1 [-] Warning: Redis
database unavailable; fingerprints will be reported as unknown. [-] Captured ClientHello I Flow: [10.7.15.147]:53802 -> [20.207.73.82]:443 |
Reassembled Size: 1533 bytes | [V-Version] 0x0303 | [V-SNI] github.com | [V-ALPN] h2 | [V-Ciphers] Count: 20
[0x1302, 0x1303, 0x1301, 0xc02c, 0xc02b, 0xc02a, 0xc030, 0xc02f, 0xc028, 0xc00a, 0xc009, 0xc014, 0xc013, 0x009d, 0x009c, 0x0035,
0x002f, 0xc008, 0xc012, 0x000a] | [V-Extensions] Count: 13 [0x0000, 0x0017, 0xff01, 0x000a, 0x000b, 0x0010, 0x0005, 0x000d, 0x0012,
0x0033, 0x002d, 0x002b, 0x001b] | [V-Curves] Count: 5 [0x11ec, 0x001d, 0x0017, 0x0018, 0x0019] | [V-SigAlgs] Count: 10
[0x0403, 0x0804, 0x0401, 0x0503, 0x0805, 0x0805, 0x0501, 0x0806, 0x0601, 0x0201] | [V-PreHash-JA3] 771,4866-4867-4865-49196-
49195-52393-49200-49199-52392-49162-49161-49172-49171-157-156-53-47-49160-49170-10,0-23-65281-10-11-16-5-13-18-51-45-43-
27,4588-29-23-24-25,0 | [V-PreHash-
JA4b]000a,002f,0035,009c,009d,1301,1302,1303,c008,c009,c00a,c012,c013,c014,c02b,c02c,c02f,c030,cca8,cca9 | [V-PreHash-
JA4c]0005,000a,000b,000d,0012,0017,001b,002b,002d,0033,ff01_0403,0804,0401,0503,0805,0805,0501,0806,0601,0201 | [SNI]
github.com | [JA3 String] 771,4866-4867-4865-49196-49195-52393-49200-49199-52392-49162-49161-49172-49171-157-156-53-47-
49160-49170-10,0-23-65281-10-11-16-5-13-18-51-45-43-27,4588-29-23-24-25,0 | [JA3 Hash] ecdf4f49dd59effc439639da29186671
| [JA3 Match] <database unavailable> | [JA4] t13d2013h2_a09f3c656075_7f0f34a4126d | [JA4 Match] <database
unavailable> the en0 is working fine but I am on Mac and the any flag is not working for me what can be the potential issue and how to sort it.
#include "tlsfp/capture.hpp" #include <iostream> #include <unistd.h> #include <csignal> #include <filesystem> #include <system_error>
namespace fs = std::filesystem; static void print_usage(const char *prog_name) { std::cerr << "Usage: " << prog_name << "[OPTIONS]\n\n" <<
"Capture Options:\n" << "-i <interface> Live network interface (e.g., any, eth0)\n" << "-r <pcap_file> Read packets from offline PCAP file\n" << "-w
<pcap_file> Save matched TLS handshakes to output PCAP\n" << "-f <bpf_filter> Custom BPF filter expression (default: 'tcp')\n" <<
"Output Mode Options:\n" << "-q Quiet mode: suppress all per-packet I/O, output benchmark summary\n" << "-v Verbose mode: print detailed
dissection, framing, and pre-hash strings\n" << "-h Show this help message\n" << "Examples:\n" << "Benchmark: " << prog_name << "-r
trace.pcap -q\n" << "Deep Debug: " << prog_name << "-r trace.pcap -v\n" << "Live Capture: " << prog_name << "-i any -f 'tcp port 443'\n"; }
int main(int argc, char *argv[]) { if (argc < 2) { print_usage(argv[0]); return 1; } tlsfp::CaptureOptions opts; int opt; while ((opt = getopt(argc, argv,
"i:r:w:f:qvh")) != -1) { switch (opt) { case 'i': opts.interface_name = optarg; break; case 'r': opts.read_filename = optarg; break; case 'w':
opts.write_filename = optarg; break; case 'f': opts.bpf_filter = optarg; break; case 'q': opts.quiet = true; break; case 'v': opts.verbose = true;
break; case 'h': print_usage(argv[0]); return 0; default: print_usage(argv[0]); return 1; } // Catch rogue unparsed positional arguments if (optind
< argc) { std::cerr << "[!] Error: Unrecognized extra argument: " << argv[optind] << "\n"; print_usage(argv[0]); return 1; } // Validate mutual
exclusivity if (opts.interface_name.empty() && opts.read_filename.empty()) { std::cerr << "[!] Error: Must specify an interface (-i) or PCAP file (-
r).\n"; print_usage(argv[0]); return 1; } if (!opts.interface_name.empty() && !opts.read_filename.empty()) { std::cerr << "[!] Error: Cannot specify
both -i (live) and -r (file) at the same time.\n"; print_usage(argv[0]); return 1; } // Prevent self-destructive read/write overlap if
(!opts.read_filename.empty() && opts.read_filename == opts.write_filename) { std::cerr << "[!] Error: Input and output PCAP file paths cannot be
identical.\n"; return 1; } // Exception-safe output path resolution if (!opts.write_filename.empty()) { fs::path p(opts.write_filename); if
(!p.has_parent_path()) { std::error_code ec; fs::path target_dir = "../pcaps"; // Only redirect to ../pcaps if that directory structure actually
exists if (fs::exists(target_dir, ec)) { opts.write_filename = (target_dir / p).string(); } } // POSIX sigaction for deterministic interrupt safety struct
sigaction sa{}; sa.sa_handler = tlsfp::signal_handler; sigemptyset(&sa.sa_mask); sa.sa_flags = 0; // Avoid SA_RESTART so pcap_loop/blocking
calls unblock sigaction(SIGINT, &sa, nullptr); sigaction(SIGTERM, &sa, nullptr); if (!tlsfp::start_capture(opts)) { std::cerr << "[!] Capture engine
encountered a fatal error.\n"; return 1; } return 0; }
```

Response: macOS uses **libpcap / BPF (BSD Packet Filter)**, which does not natively support Linux's synthetic "any" pseudodevice.

When you pass "any" on macOS, `pcap_open_live()` either fails silently or creates a binding with datalink type 12 (DLT\_RAW or DLT\_LOOP), where standard Ethernet frame headers (DLT\_EN10MB) are missing. As a result, your packet parser fails to unpack Link/Network headers and misses incoming packets.

## Root Causes on macOS

- Missing any Pseudodevice Support:** Unlike Linux's `packet(7)` sockets, macOS BPF devices must bind to a specific physical or virtual network interface (`en0`, `en1`, `lo0`, etc.).
- Datalink Type Mismatch (DLT\_RAW vs DLT\_EN10MB):** Notice the difference in your logs:
  - `-i any`: Output shows `Datalink type: 12`. Your Ethernet parser breaks because packets don't contain standard L2 Ethernet headers.
  - `-i en0`: Output shows `Datalink type: 1 (DLT_EN10MB)`. Standard Ethernet parsing succeeds.

## Solutions

Choose one of the following implementations based on how you want your application to behave.

### Option 1: Map "any" to nullptr (Recommended)

`libpcap` provides native cross-platform support for capturing on all interfaces when `nullptr` (or `PCAP_IF_ANY`) is passed to `pcap_open_live` or `pcap_create`.

Update `tlsfp::start_capture` inside your C++ library:

```
// Inside tlsfp::start_capture implementation
const char* dev = opts.interface_name.c_str();

// Map "any" to nullptr for libpcap multi-interface capture
if (opts.interface_name == "any") {
    dev = nullptr; // Or "any" on Linux, but nullptr works universally across modern libpcap versions
}

char errbuf[PCAP_ERRBUF_SIZE];
```

```
pcap_t* handle = pcap_open_live(dev, BUFSIZ, 1, 1000, errbuf);
```

## Option 2: Validate and Guard the Interface in `main.cpp`

If you want to prevent users from executing invalid configurations on macOS, resolve or block "any" directly inside your CLI parser:

```
#ifdef __APPLE__
    if (opts.interface_name == "any") {
        std::cerr << "[-] Error: 'any' interface is not supported on macOS.\n"
        << " [*] Please specify a concrete interface (e.g., -i en0) or use pcap_findalldevs().\n"
        return 1;
    }
#endif
```

## Option 3: Dynamically Support Datalink Type 12 (`DLT_RAW`)

If you intend to keep `Datalink type: 12` support for captures that bypass L2 headers (such as raw IP interfaces, VPN tun devices, or loopback interfaces):

Check the datalink type in your packet parsing handler:

```
int dlt = pcap_datalink(pcap_handle);

if (dlt == DLT_EN10MB) {
    // Skip 14 bytes of Ethernet header
    ip_header = (struct ip*)(packet_bytes + 14);
} else if (dlt == DLT_RAW || dlt == 12) {
    // No L2 header present; payload starts directly with IP header
    ip_header = (struct ip*)(packet_bytes);
}
```

*User prompt: I want the any functionality of linux in my mac*

Response: To replicate Linux's `any` interface behavior on macOS using standard `libpcap`, you can either configure `pcap_create` to capture across all devices or enumerate all active non-loopback interfaces dynamically with `pcap_findalldevs()`.

Below are the two standard approaches to implement multi-interface capture on macOS in C++.

## Method 1: Cross-Platform `pcap_create` Mode (Simplest)

Modern versions of `libpcap` on macOS support an implicit "all interfaces" capture when passing `nullptr` or "any" through the `pcap_create` API family rather than the legacy `pcap_open_live`.

### 1 Initialize Handle with NULL target

`pcap_create`

Pass `nullptr` (or check if device name is "any") to open an implicit global handle:

```
const char* dev = (opts.interface_name == "any") ? nullptr : opts.interface_name.c_str();

char errbuf[PCAP_ERRBUF_SIZE];
pcap_t* handle = pcap_create(dev, errbuf);
if (!handle) {
    std::cerr << "[-] Error creating pcap handle: " << errbuf << "\n";
    return false;
}
```

### 2 Set Capture Options & Activate

Configuration

Configure snapshot length, promiscuous mode, and immediate mode before calling `pcap_activate`:

```
pcap_set_snaplen(handle, 65535);
pcap_set_promisc(handle, 1);
pcap_set_timeout(handle, 1000);
pcap_set_immediate_mode(handle, 1); // Crucial on macOS to reduce buffer latency

int status = pcap_activate(handle);
if (status < 0) {
    std::cerr << "[-] Error activating handle: " << pcap_geterr(handle) << "\n";
    pcap_close(handle);
    return false;
}
```

### 3 Verify Link Type

Verification

Verify the datalink type returned by `pcap_datalink(handle)`. If capturing globally on macOS, it often returns `DLT_RAW (12)` or `DLT_NULL (0)`. Ensure your frame offsets skip Layer 2 headers dynamically based on `pcap_datalink()`.

## Method 2: Multi-Threaded Interface Binding (True Linux any Parity)

Because macOS BPF sockets isolated to individual hardware interface drivers (like `en0`, `en1`, `ap0`), true Linux parity across distinct physical drivers is typically achieved by spawning a capture worker thread for each active interface returned by `pcap_findalldevs()`.

### 1 Enumerate Active Interfaces

`pcap_findalldevs`

Query all system interfaces using `pcap_findalldevs()`:

```
pcap_if_t *alldevs, *dev;
char errbuf[PCAP_ERRBUF_SIZE];

if (pcap_findalldevs(&alldevs, errbuf) == -1) {
    std::cerr << "[~] Error finding devices: " << errbuf << "\n";
    return false;
}

std::vector<std::string> target_interfaces;
for (dev = alldevs; dev != nullptr; dev = dev->next) {
    // Skip loopback or down interfaces
    if (dev->flags & PCAP_IF_LOOPBACK) continue;
    if (!(dev->flags & PCAP_IF_UP)) continue;

    target_interfaces.push_back(dev->name);
}
pcap_freealldevs(alldevs);
```

### 2 Spawn Worker Threads

Concurrency

Loop through each interface in `target_interfaces`, spawn a dedicated thread per interface, and execute `pcap_open_live` on each bound interface individually.

### 3 Aggregate Packet Streams

Processing

Pass packet pointers from all thread callbacks into a shared lock-free queue or process them centrally with your TLS fingerprinting worker routines.